

DEPARTMENT OF COMPUTER ENGINEERING

GPTU MENTOR & COUNSELLING HUB

PROJECT REPORT

2024-2025

SUBMITTED BY

DURAIMURUGAN M 23590563

HARIGARAN S 23590564

VIGNESH T 23501415



Under the guidance of

Mr. P. RAJAKUMAR

A report as partial fulfillment of the requirement of the award of

DIPLOMA IN COMPUTER ENGINEERING

of the Board of Technical Education, Tamil Nadu.

139, GOVERNMENT POLYTECHNIC COLLEGE

UTHANGARAI – 635 207

139, GOVERNMENT POLYTECHNIC COLLEGE
UTHANGARAI – 635 207



BONAFIDE CERTIFICATE

This is to certify that the project work entitled “**GPTU MENTOR & COUNSELLING HUB**” being submitted by **VIGNESH T – 23501415** as a partial fulfillment for the requirement of award of Diploma in Computer Engineering by **DOTe** in the year **2024-2025**.

Head of the department

Project Guide

Project Coordinator

Certified that the candidate was examined in the viva – voce examination held at **GOVERNMENT POLYTECHNIC COLLEGE, UTHANGARAI** on _____

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our beloved **PRINCIPAL Mr. M. VIJAYAN, M.Sc., M.Phil., B.Ed.**, for permitting us to do this project and helping us by providing required facilities and necessary resources to complete our project work successfully.

We convey our profound gratitude to our beloved **Dr. P. EZHILARASU, B.E., M.E., Ph.D., HEAD OF THE DEPARTMENT (i/e)**, for his help and encouragement given to us for this project.

We pay valuable note of thanks to our beloved guide **Mr. P. RAJAKUMAR** for his valuable guidance and technical advice given during the period of difficulties and to his helpful co- operation towards completion of our project “**GPTU MENTOR & COUNSELLING HUB**”.

We express our gratitude to our department staff members **Dr. EZHILARASU P, Mr. RAJAKUMAR P, Mr. SATHIYAMOORTHY M, Mr. NAGOOR HANIFA G and Mr. THANIGESAN M** for their invaluable encouragement in completing this project work.

Finally, we wish to express our heartfelt thanks to our staffs, our family members and friends for their kind co-operation by means of valuable suggestions and timely help.

TABLE OF CONTENTS

ABSTRACT.....	05
1. INTRODUCTION.....	06
2. PROBLEM DEFINITION.....	06
3. EXISTING SYSTEM.....	07
4. PROPOSED SYSTEM.....	07
5. MODULES.....	08
5.1 ADMIN.....	08
5.2 STAFF.....	08
5.3 STUDENT.....	08
6. SYSTEM REQUIREMENTS.....	09
6.1 HARDWARE REQUIREMENTS.....	09
6.2 SOFTWARE REQUIREMENTS.....	09
7. FEATURES.....	10
7.1 ADMIN FEATURES.....	10
7.2 STAFF FEATURES.....	10
7.3 STUDENT FEATURES.....	11
8. DIAGRAMS.....	12
8.1 SYSTEM DIAGRAM.....	12
8.2 ADMIN LOGIN ACTIVITY DIAGRAM.....	13
8.3 STAFF LOGIN ACTIVITY DIAGRAM.....	14
9. DATABASES.....	15
10. SOURCE CODE.....	17
11. RESULT EVALUATION.....	31
12. FUTURE SCOPE.....	33
13. CONCLUSION.....	33
14. REFERENCE.....	34

ABSTRACT

The **GPTU Mentor & Counselling Hub** is a web application designed to help colleges manage student information in one place. It keeps track of personal, academic, and non-academic details of students. Staff members can use this system to view student records, give counselling support, and solve student-related problems quickly and easily. The goal is to make the process smoother and reduce the use of paper by going fully digital.

This system is built using Django and follows a modular design, meaning each part of the project (like user management, student records, and data search) is separated and easy to manage. It also has secure login features, a simple user interface, and options for searching and updating student data. Overall, the project helps improve communication between students and staff while saving time and effort.

1. INTRODUCTION

In today's educational environment, managing student information and providing timely support is more important than ever. Traditional methods that rely on paperwork or scattered digital tools often lead to confusion, delays, and a lack of proper communication between students and staff. To solve these challenges, institutions need a smart, centralized system that can handle student records and support services in one place.

The **GPTU Mentor & Counselling Hub** is developed to meet this need. It is a web-based application that allows colleges to manage student data, track academic and non-academic progress, and offer counselling support efficiently. Built using Django, this system provides a structured way to store and retrieve information securely. It also makes it easy for staff to guide students, monitor their development, and solve issues quickly through a simple and user-friendly interface.

2. PROBLEM DEFINITION

In many colleges, handling student information is a big challenge. Most of the time, data is stored in different places or written on paper, which makes it hard to find the right information when needed. Teachers and staff waste a lot of time looking for student records, and sometimes important details are missed or lost.

Also, there is no single system where all student information like personal details, marks, and counselling notes is available together. Because of this, students might not get proper help or support on time. This can lead to more problems in their studies and personal life. A better, organized system is needed to solve these issues easily and quickly.

3. EXISTING SYSTEM

In most colleges, student data is managed using manual records or basic digital tools like Excel sheets. Information such as personal details, academic performance, and counselling notes are often stored separately in different departments. This makes it hard for staff to access complete information about a student quickly.

Also, there is usually no proper system for tracking student counselling or support services. Communication between students and staff is mostly offline or through emails, which can be slow and unorganized. These methods are time-consuming, can lead to errors, and make it difficult to provide timely help to students when they need it the most.

4. PROPOSED SYSTEM

The **GPTU Mentor & Counselling Hub** is a web-based system that solves the problems of the existing system by bringing all student data into one platform. It allows staff to manage personal, academic, and non-academic information of students in a simple and organized way. Everything is stored in a single database, so staff can easily access complete student records anytime.

The system also includes features for providing counselling support, tracking student issues, and keeping a record of all actions taken. It is built using Django, which makes it secure, fast, and easy to maintain. With this system, colleges can save time, avoid paperwork, improve communication, and give students the support they need in a timely manner.

5. MODULES

5.1. ADMIN

The admin has full control over the system. These modules help the admin manage users and maintain overall system functionality.

- **User Approval Module**
Admin can approve or reject registration requests from staff users.
- **Role & Access Control Module**
Assign roles (admin, staff, student) and manage permissions.
- **Staff Management Module**
View, edit, or remove staff accounts when needed.
- **System Monitoring Module**
Monitor overall usage and ensure that data is handled properly and securely.

5.2. STAFF

Staff members handle students, view records, provide counselling, and track student progress.

- **Student Management Module**
Add new student records, view student details and edit or delete student records.
- **Feedback & Issue Handling Module**
View and manage issues or feedback raised by students.
- **Search & Filter Module**
Quickly find students using filters like name, department, year, or register number.

5.3. STUDENT

Students have limited access in the system. Their modules are focused only on viewing their data and giving feedback.

- **Profile Viewing Module**

Students can view their personal, academic, and non-academic details stored in the system with the permission of their mentor.

- **Feedback Module**

Students can submit feedback regarding their academic experience, issues they face, or suggestions for improvement.

6. SYSTEM REQUIREMENTS

6.1. HARDWARE REQUIREMENTS

Components	Specification
Processor	: Intel i3 or above
RAM	: Minimum 4GB
Hard Disk	: At least 250 GB free space
Monitor	: 1024×768 resolution or higher

6.2. SOFTWARE REQUIREMENTS

Components	Specification
Operating System	: Windows 10 or Windows 11
Backend Framework	: Django (Python)
Frontend	: HTML, CSS, JavaScript
Database	: MySQL
PDF Library	: PDF report export
Browser	: Chrome / Firefox / Edge
Editor	: VS Code

7. FEATURES

7.1. ADMIN

The admin is the main controller of the system. They manage staff users and keep the system organized and secure.

- **Approve Staff Accounts:** Admin can approve or reject staff registration requests.
- **Manage Roles and Permissions:** They decide who can access what in the system.
- **View and Edit Staff Details:** Admin can update or remove staff profiles if needed.
- **Monitor System Usage:** Admin can keep an eye on how the system is being used.
- **Secure Login:** Only authorized admins can access the admin panel.

7.2. STAFF

Staff are the main users who interact with student data and support students.

- **Manage Students:** Staff can add, approve, edit, or delete student records in their department.
- **Search Students Easily:** They can search and filter student details using different options.
- **View Student Data:** They can access each student's academic and personal details.
- **Check Feedback:** Staff can read the feedback submitted by students.
- **Export Data to PDF:** Staff can download student or counselling records in PDF format.
- **Secure Login:** Each staff member has a secure login to access their dashboard.

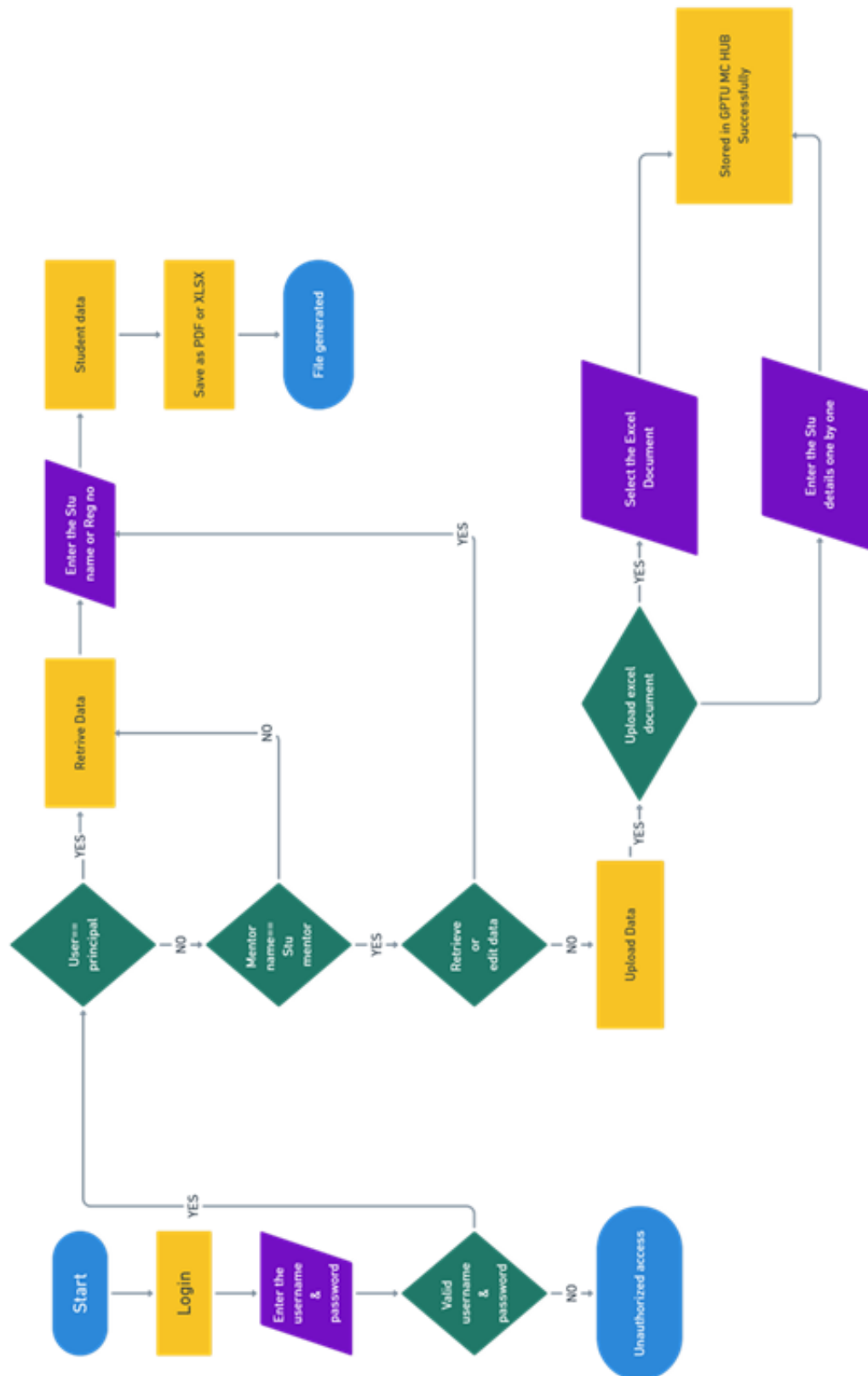
7.3. STUDENT

Students have limited access and can only view their information and give feedback.

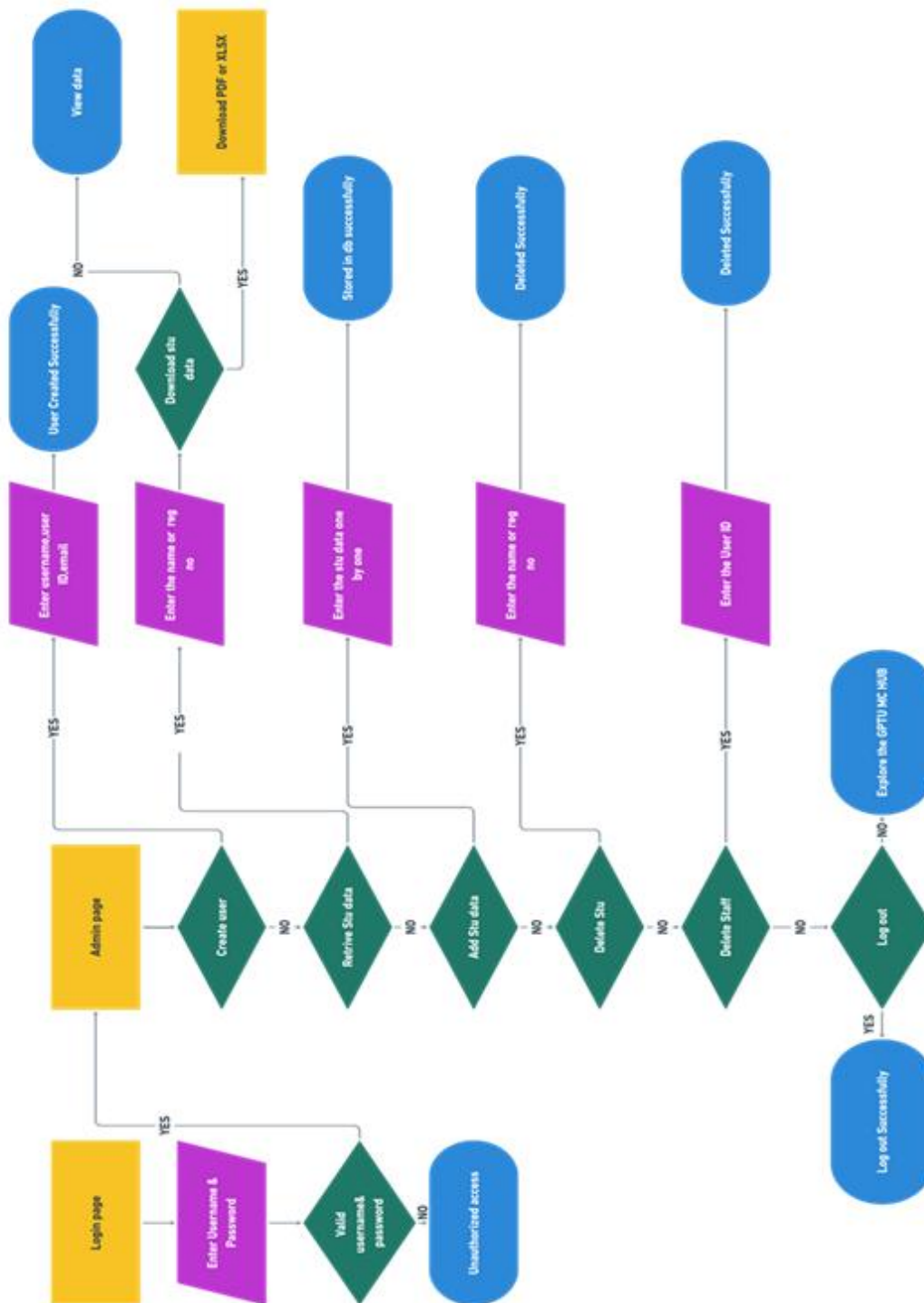
- **View Profile:** Students can see their personal, academic, and counselling-related data.
- **Submit Feedback:** They can share feedback about their experience or any issues.
- **Secure Login:** Students can log in securely to view their dashboard.

8. DIAGRAMS

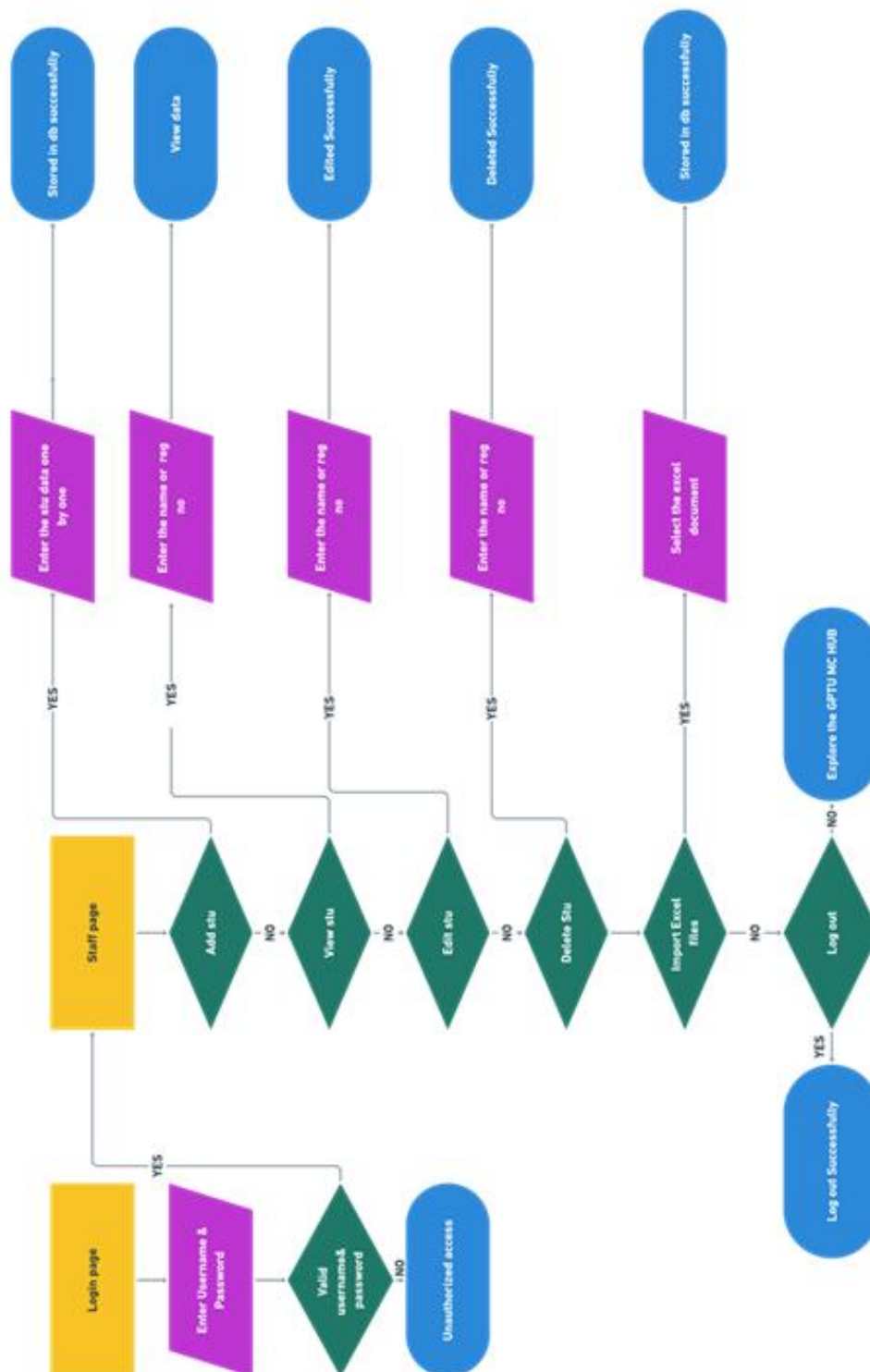
8.1. SYSTEM DIAGRAM



8.2. ADMIN LOGIN ACTIVITY DIAGRAM

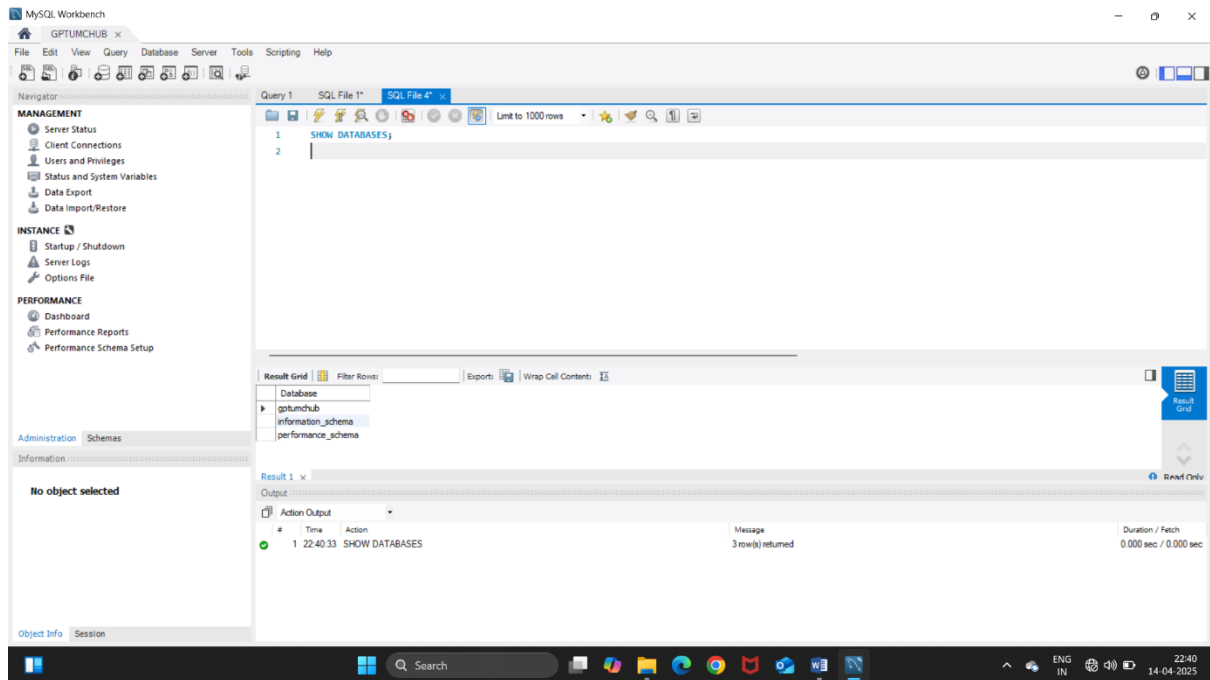


8.3. STAFF LOGIN ACTIVITY DIAGRAM

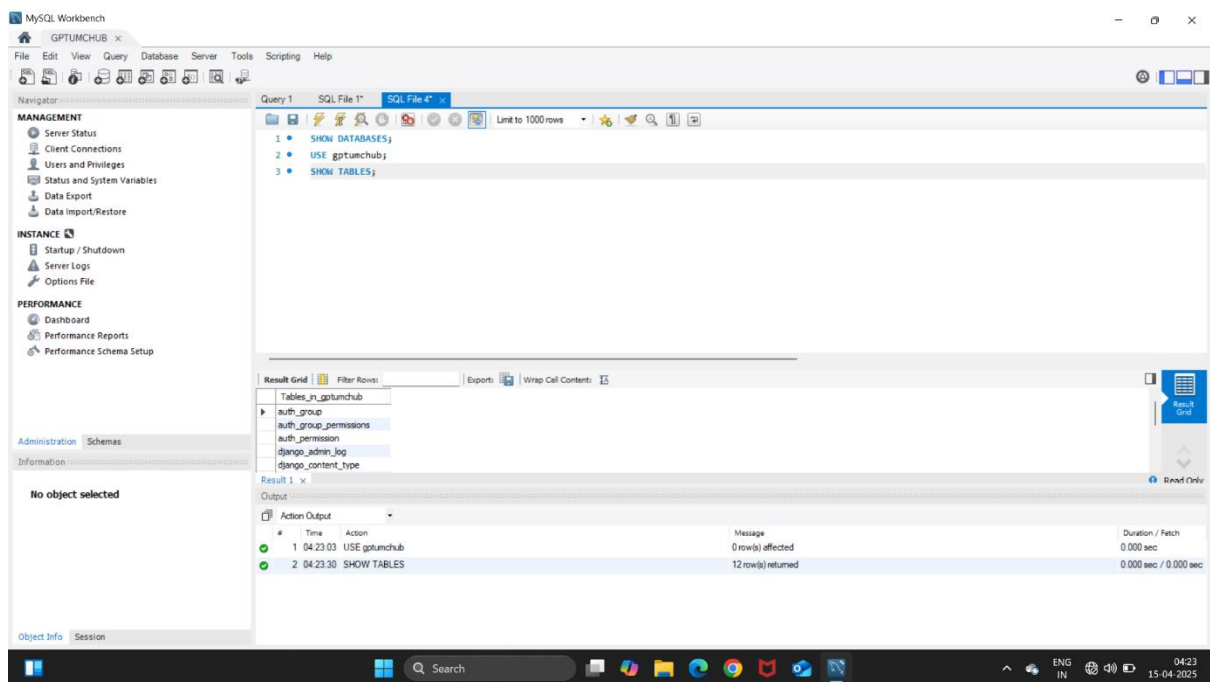


9. DATABASES

DATABASE NAME: GPTUMCHUB



TABLES USED IN GPTUMCHUB



MySQL Workbench

GPTUMCHUB

File Edit View Query Database Server Tools Scripting Help

Navigator

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Administration Schemas

Information

No object selected

Object Info Session

Query 1 SQL File 1* SQL File 4*

Limit to 1000 rows

```

1 SHOW DATABASES;
2 USE gptumchub;
3 SHOW TABLES;
4 SELECT * FROM home_userprofile;

```

Result Grid

#	id	last_login	is_superuser	is_staff	is_active	date_joined	first_name	last_name	email	user_id	username	password
11	2025-04-10 15:01:22.388478	1	1	1	2025-04-01 14:53:08.000000	GPTU MC HUB	Administrator	gptumchub@gmail.com		139	admin	pbkdf2_sha256\$8700000
29	2025-04-13 04:39:11.143015	0	0	1	2025-04-02 15:20:54-01 14:53:08.000000		Thilagaraj	vigneshthilagaraj00@gmail.com		23501415	@vignesh	pbkdf2_sha256\$8700000
30		0	0	1	2025-04-07 18:04:42.465035							pbkdf2_sha256\$8700000

home_userprofile 2 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	04:23:03	USE gptumchub	0 row(s) affected	0.000 sec
2	04:23:30	SHOW TABLES	12 row(s) returned	0.000 sec / 0.000 sec
3	04:24:04	SELECT * FROM home_userprofile LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec

04:24 15-04-2025

MySQL Workbench

GPTUMCHUB

File Edit View Query Database Server Tools Scripting Help

Navigator

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Administration Schemas

Information

No object selected

Object Info Session

Query 1 SQL File 1* SQL File 4*

Limit to 1000 rows

```

1 SHOW DATABASES;
2 USE gptumchub;
3 SHOW TABLES;
4 SELECT * FROM home_userprofile;
5 SELECT * FROM users_student;

```

Result Grid

#	id	first_name	last_name	reg_no	program_name	program_type	year_of_joining	mentor_name	father_name	mother_name	dob	gender	blood_group	mother_tongue	differently
1	Vignesh	T	23501415	DCSE	Regular	2022-07-21	Vignesh	Thilagaraj	Vijaya	2006-07-21	Male	O+	Telugu	No	
2	Student	GPTU		DCSE	Regular	2022-07-21	User	Harshath	Kannagi	2006-07-21	Others		Tamil	No	
3	Harigan	S	23590564	DCSE	Lateral	2023-12-12	DURAIMURUGAN	Salthivel		2004-09-05	Male	O+	Tamil	No	
6	Murali	M		DMK	Regular	2021-07-21	Vignesh	Manikandan	Malga	2004-05-21	Male	A+	Tamil	No	

users_student 3 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	04:23:03	USE gptumchub	0 row(s) affected	0.000 sec
2	04:23:30	SHOW TABLES	12 row(s) returned	0.000 sec / 0.000 sec
3	04:24:04	SELECT * FROM home_userprofile LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
4	04:24:48	SELECT * FROM users_student LIMIT 0, 1000	15 row(s) returned	0.031 sec / 0.000 sec

04:25 15-04-2025

10. SOURCE CODE

Views.py

```
from django.shortcuts import render, redirect
from django.contrib import messages
from .models import UserProfile, PasswordResetToken
from home.models import UserProfile, PasswordResetToken
from django.contrib.auth.hashers import make_password
from django.utils.timezone import now, timedelta
from django.core.mail import EmailMessage
from django.conf import settings
from django.contrib.auth import authenticate, login as auth_login
from django.views.decorators.cache import never_cache
from django.utils.cache import add_never_cache_headers
from django.core.mail import EmailMultiAlternatives
import uuid
import string
import random
import datetime
import logging

# Create your views here.

# To render the Home page of the web application (index.html)
def index(request):
    return render(request, 'home/index.html')

logger = logging.getLogger(__name__)

# To render the Log-In page file
@never_cache
def login(request):
    if request.method == 'POST':
        user_input = request.POST.get('user_input') # Input can be
        user_id or username
        password = request.POST.get('password')

        logger.info(f"Login attempt: user_input={user_input},
        password={password}")

        # Server-side validation
```

```

        if not user_input or not password:
            messages.error(request, 'Both username/user ID and
password are required.')
            return redirect('home:login')

        user_by_username = authenticate(request,
username=user_input, password=password)
        user_by_userid = authenticate(request, user_id=user_input,
password=password)

        user = user_by_username or user_by_userid

        # Try authenticating with user_id or username
        #user = authenticate(request, username=user_input,
password=password)

        if user:
            auth_login(request, user) # Django manages session
automatically
            logger.info(f"Login successful: user={user}")

            return redirect('users:dashboard')
        else:
            messages.error(request, 'Invalid username/user ID or
password.')
            logger.warning(f"Login failed: user_input={user_input}")

            return redirect('home:login')

    response = render(request, 'home/login.html')
    add_never_cache_headers(response) # Prevents browser from
storing login page
    return response

# To render the Forgot Password page
# Used to change the users password if the user forgot their
password
def forgot_password(request):
    if request.method == "POST":
        email = request.POST.get('email').strip()

        try:
            user = UserProfile.objects.get(email=email)
            token = str(uuid.uuid4()) # Generate unique token

```

```

# Save reset token in database
PasswordResetToken.objects.create(user=user,
token=token)

# Create reset link
reset_link = f"http://{settings.LOCAL_IP}:8000/reset-
password/{token}/"

subject = "GPTU MC HUB - Password Reset Request"
from_email = f"GPTU MC HUB <{settings.EMAIL_HOST_USER}>"
text_content = f"Hello {user.first_name},\n\nWe received
a request to reset your password.\nClick the following link to reset
your password:\n{reset_link}\n\nIf you did not request this, please
ignore this email.\n\nRegards,\nGPTU MC HUB Team"

html_content = f"""
<html>
<body style="font-family: Arial, sans-serif; color:
#333; padding: 20px;">
    <h2>GPTU MC HUB - Password Reset Request</h2>
    <p>Hello <strong>{user.first_name}</strong>,</p>
    <p>We received a request to reset your password.
Please click the button below to reset it:</p>
    <p style="margin: 30px 0;">
        <a href="{reset_link}" target="_blank"
            style="background-color: #2c3e50; color:
#fff; padding: 12px 20px; text-decoration: none; border-radius: 5px;
font-size: 16px; display: inline-block;">
            Reset Password
        </a>
    </p>
    <p>If the button above does not work, you can also
copy and paste this link in your browser:</p>
    <p><a href="{reset_link}"
target="_blank">{reset_link}</a></p>
    <br>
    <p>If you did not request a password reset, please
ignore this email or contact support.</p>
    <br>
    <p>Regards,</p>
    <p style="font-weight: bold; color: #2c3e50;">GPTU
MC HUB Team</p>
<hr>

```

```

        <small>This is an automated email; please do not
reply.</small>
    </body>
</html>
"""

    # Send HTML email
    msg = EmailMultiAlternatives(subject, text_content,
from_email, [email])
    msg.attach_alternative(html_content, "text/html")
    msg.send()

    messages.success(request, "Password reset link sent to
your email.")
    return redirect('home:forgot_password')
except UserProfile.DoesNotExist:
    messages.error(request, "Email not found.")
    return redirect('home:forgot_password')

return render(request, 'home/forgot_password.html')

# To render the Reset Password page according to the reset link
which is sent to user's corresponding E-Mail ID
def reset_password(request, token):
    try:
        reset_token = PasswordResetToken.objects.get(token=token)

        # Check if token is expired (valid for 15 minutes)
        if now() - reset_token.created_at > timedelta(minutes=15):
            messages.error(request, "Password reset link expired.")
            return redirect('home:forgot_password')

        if request.method == "POST":
            new_password = request.POST.get('password')
            confirm_password = request.POST.get('confirm_password')

            if new_password != confirm_password:
                messages.error(request, "Passwords do not match.")
                return render(request, 'home/reset_password.html',
{"token": token})

            if len(new_password) < 8:

```

```

        messages.error(request, "Password must be at least 8
characters.")
        return render(request, 'home/reset_password.html',
{"token": token})

        special_chars = set(string.punctuation)
        if not any(char in special_chars for char in
new_password):
            messages.error(request, 'Password must contain at
least one special character!')
            return render(request, 'home/reset_password.html',
{"token": token})

        # Update user password
        user = reset_token.user
        user.password = make_password(new_password)
        user.save()

        # Delete token after successful reset
        reset_token.delete()

        messages.success(request, "Password reset
successfully.")
        return redirect('home:login')

    except PasswordResetToken.DoesNotExist:
        messages.error(request, "Invalid or expired reset link.")
        return redirect('home:forgot_password')

    return render(request, 'home/reset_password.html', {"token":
token})

# Function used to Generate and send the 6 digit OTP to users
corresponding E-Mail ID
def send_otp(email, request):
    otp = ''.join(random.choices(string.digits, k=6)) # Generate 6-
digit OTP
    expiry_time = datetime.datetime.now() +
datetime.timedelta(minutes=1)
    request.session['otp'] = otp
    request.session['otp_expiry'] = expiry_time.timestamp()

    subject = "Your GPTU MC HUB Email Verification Code"
    from_email = f"GPTU MC HUB <{settings.EMAIL_HOST_USER}>"

```

```

text_content = f"Hello,\n\nYour OTP for email verification is:
{otp}\nThis OTP will expire in 1 minute.\n\nPlease do not share this
code with anyone.\n\nRegards,\nGPTU MC HUB Team"

```

```

html_content = f"""
<html>
<body style="font-family: Arial, sans-serif; color: #333;">
  <h2>GPTU MC HUB - Email Verification</h2>
  <p>Hello,</p>
  <p>Your One-Time Password (OTP) for verification is:</p>
  <h1 style="color: #2c3e50;">{otp}</h1>
  <p>This OTP will expire in <strong>1 minute</strong>.</p>
  <p>Please do not share this code with anyone.</p>
  <br>
  <p>Regards,</p>
  <p style="color: #2c3e50; font-weight: bold;">GPTU MC HUB
Team</p>
  <hr>
  <small>This is an automated email; please do not
reply.</small>
</body>
</html>
"""

```

```

msg = EmailMultiAlternatives(subject, text_content, from_email,
[email])
msg.attach_alternative(html_content, "text/html")
msg.send()
return otp

```

To render the Signup page

```
def signup(request):
```

```
    if request.method == "POST":
```

```

        # To get the input values from the users to create an
account

```

```

        first_name = request.POST.get('firstName', '').strip()
        last_name = request.POST.get('lastName', '').strip()
        email = request.POST.get('email', '').strip()
        user_id = request.POST.get('userId', '').strip()
        username = request.POST.get('username', '').strip()
        password = request.POST.get('password', '').strip()

```

```

        confirm_password = request.POST.get('confirmPassword',
        '').strip()
        department = request.POST.get('department', '').strip()
        gender = request.POST.get('gender', '').strip()

        # Check if any field is empty
        if not all([first_name, last_name, email, user_id, username,
password, confirm_password, department, gender]):
            messages.error(request, 'All fields are required! Please
fill in all fields.')
            return render(request, 'home/signup.html')

        # To Ensure the given username is starts with '@'
        if username[0] != '@':
            messages.error(request, "The Username must start with
'@'")

            return render(request, 'home/signup.html', {
                'first_name': first_name,
                'last_name': last_name,
                'email': email,
                'user_id': user_id,
                'department': department,
                'gender': gender,
            })

        # To Ensure the given username contains only lowercase
letters
        if not username.islower():
            messages.error(request, 'Username must contain only
lowercase letters.')
            return render(request, 'home/signup.html', {
                'first_name': first_name,
                'last_name': last_name,
                'email': email,
                'user_id': user_id,
                'department': department,
                'gender': gender,
            })

        # To check the Password and Confirm Password both are same
or not
        if password != confirm_password:
            messages.error(request, 'Password do not match!')

```

```

        return render(request, 'home/signup.html', {
            'first_name': first_name,
            'last_name': last_name,
            'email': email,
            'user_id': user_id,
            'username': username,
            'department': department,
            'gender': gender,
        })

    # To Ensure the given password has at least 8 charactes
    if len(password) < 8:
        messages.error(request, 'Password length must be at
least 8 characters!')
        return render(request, 'home/signup.html', {
            'first_name': first_name,
            'last_name': last_name,
            'email': email,
            'user_id': user_id,
            'username': username,
            'department': department,
            'gender': gender,
        })

    # To Ensure the given password contains at least one special
character
    special_chars = set(string.punctuation)
    if not any(char in special_chars for char in password):
        messages.error(request, 'Password must contain at least
one special character!')
        return render(request, 'home/signup.html', {
            'first_name': first_name,
            'last_name': last_name,
            'email': email,
            'user_id': user_id,
            'username': username,
            'department': department,
            'gender': gender,
        })

    # Check if username already exists
    if UserProfile.objects.filter(username=username).exists():
        messages.error(request, "Username already taken. Please
choose another one.")

```



```

        return render(request, 'home/signup.html', {
            'first_name': first_name,
            'last_name': last_name,
            'email': email,
            'user_id': user_id,
            'department': department,
            'gender': gender,
        })

    # Check if user_id already exists
    if UserProfile.objects.filter(user_id=user_id).exists():
        messages.error(request, "User ID already exists.")
        return render(request, 'home/signup.html', {
            'first_name': first_name,
            'last_name': last_name,
            'email': email,
            'username': username,
            'department': department,
            'gender': gender,
        })

    """

    # Validate user_id and email against auth_users
    if not AuthUser.objects.filter(user_id=user_id,
email=email).exists():
        messages.error(request, "You're not eligible to create
an account!")
        return render(request, 'home/signup.html')
    """

    # Calling the send_otp() to send the otp if the given inputs
are correct
    otp = send_otp(email, request)
    request.session['otp'] = otp
    request.session['user_data'] = {
        'first_name': first_name,
        'last_name': last_name,
        'email': email,
        'user_id': user_id,
        'username': username,
        'department': department,
        'gender': gender,
        'password': password, # Store plain password
temporarily for user creation

```

```

    }

    return redirect('home:verify_otp') # Redirect to OTP
verification page

    return render(request, 'home/signup.html')

# To verify the otp
def verify_otp(request):
    if request.method == "POST":
        entered_otp = request.POST.get("otp", "").strip()
        stored_otp = request.session.get("otp")
        otp_expiry = request.session.get("otp_expiry")

        # Validate OTP existence and expiration
        if not stored_otp or not otp_expiry or now().timestamp() >
otp_expiry:
            messages.error(request, "OTP has expired! Please request
a new one.")
            return redirect("home:verify_otp")

        # Validate OTP match
        if entered_otp == stored_otp:
            user_data = request.session.get("user_data", {})

            if user_data:
                try:
                    # Create user in Django authentication system
                    user = UserProfile.objects.create_user(
                        username=user_data['username'],
                        email=user_data['email'],
                        password=user_data['password'], # Django
hashes password automatically
                        first_name=user_data['first_name'],
                        last_name=user_data['last_name'],
                        user_id=user_data['user_id'],
                        department=user_data['department'],
                        gender=user_data['gender'],
                    )

                    # Clear session data
                    request.session.pop("otp", None)
                    request.session.pop("user_data", None)
                    request.session.pop("otp_expiry", None)

```

```

        messages.success(request, "Account created
successfully!")
        return redirect("home:login")

    except Exception as e:
        messages.error(request, f"Error creating
account: {e}")
        return redirect("home:verify_otp")
    else:
        messages.error(request, "Invalid OTP! Please try
again.")
        return render(request, 'home/verify_otp.html')

    return render(request, 'home/verify_otp.html')

# To resend the OTP
def resend_otp(request):
    if request.method == "POST":
        user_data = request.session.get("user_data", {})
        user_email = user_data.get("email")

        if not user_email:
            messages.error(request, "User email not found. Please
sign up again.")
            return redirect("home:verify_otp")

        # Remove Old OTP from Session
        request.session.pop("otp", None)
        request.session.pop("otp_expiry", None)

        # Generate new OTP and expiry
        new_otp = ''.join(random.choices(string.digits, k=6))
        expiry_time = datetime.datetime.now() +
datetime.timedelta(minutes=1)
        request.session["otp"] = new_otp
        request.session["otp_expiry"] = expiry_time.timestamp()
        request.session.modified = True

        # HTML email content (copied from your send_otp style)
        subject = "Your New OTP for GPTU MC HUB Verification"
        from_email = f"GPTU MC HUB <{settings.EMAIL_HOST_USER}>"

```

```

text_content = f"Hello,\n\nYour new OTP for verification is:
{new_otp}\nThis OTP will expire in 1 minute.\n\nPlease do not share
this code with anyone.\n\nRegards,\nGPTU MC HUB Team"

```

```

html_content = f"""
<html>
<body style="font-family: Arial, sans-serif; color: #333;">
    <h2>GPTU MC HUB - Email Verification</h2>
    <p>Hello,</p>
    <p>Your new One-Time Password (OTP) for verification
is:</p>
    <h1 style="color: #2c3e50;">{new_otp}</h1>
    <p>This OTP will expire in <strong>1
minute</strong>.</p>
    <p>Please do not share this code with anyone.</p>
    <br>
    <p>Regards,</p>
    <p style="color: #2c3e50; font-weight: bold;">GPTU MC
HUB Team</p>
    <hr>
    <small>This is an automated email; please do not
reply.</small>
</body>
</html>
"""

```

```

try:
    msg = EmailMultiAlternatives(subject, text_content,
from_email, [user_email])
    msg.attach_alternative(html_content, "text/html")
    msg.send()
    messages.success(request, "A new OTP has been sent to
your email.")
except Exception as e:
    messages.error(request, f"Error sending email: {e}")

```

```

return redirect("home:verify_otp")

```

```

return redirect("home:verify_otp")

```

```

#To render the Contact Us page

```

```

def contactus(request):

```

```

    if request.method == "POST":

```

```

        name = request.POST.get("name", "").strip()

```

```

email = request.POST.get("email", "").strip()
subject = request.POST.get("subject", "No Subject").strip()
message = request.POST.get("message", "").strip()

if not name:
    name = "Anonymous"
if not email:
    email = "anonymous@example.com"

email_body = f"""
Name: {name}
Email: {email}
Subject: {subject}

Message:
{message}
"""

email_send = EmailMessage(
    subject=f"Feedback from {name}",
    body=email_body,
    from_email=f"GPTU MC HUB <{settings.EMAIL_HOST_USER}>",
# Use a valid email
    to=['www.mr.comp@gmail.com'],
    reply_to=[email] # This allows replies to go to the
user's email
)

try:
    email_send.send()
    return redirect("home:contactus") # Redirect after
sending email
except Exception as e:
    print("Email sending failed:", e) # Debugging info

return render(request, 'home/contactus.html')

# To render the About Us page
def aboutus(request):
    return render(request, 'home/aboutus.html')

```

Urls.py

```

from django.urls import path

```

```

from . import views

app_name = 'home'

urlpatterns = [
    path("", views.index, name='index'),
    path("login/", views.login, name='login'),
    path("signup/", views.signup, name='signup'),
    path("forgot-password/", views.forgot_password,
name='forgot_password'),
    path('reset-password/<str:token>', views.reset_password,
name='reset_password'),
    path("contact-us/", views.contactus, name='contactus'),
    path("about-us/", views.aboutus, name='aboutus'),
    path("verify-otp/", views.verify_otp, name='verify_otp'),
    path('resend-otp/', views.resend_otp, name='resend_otp'),
]

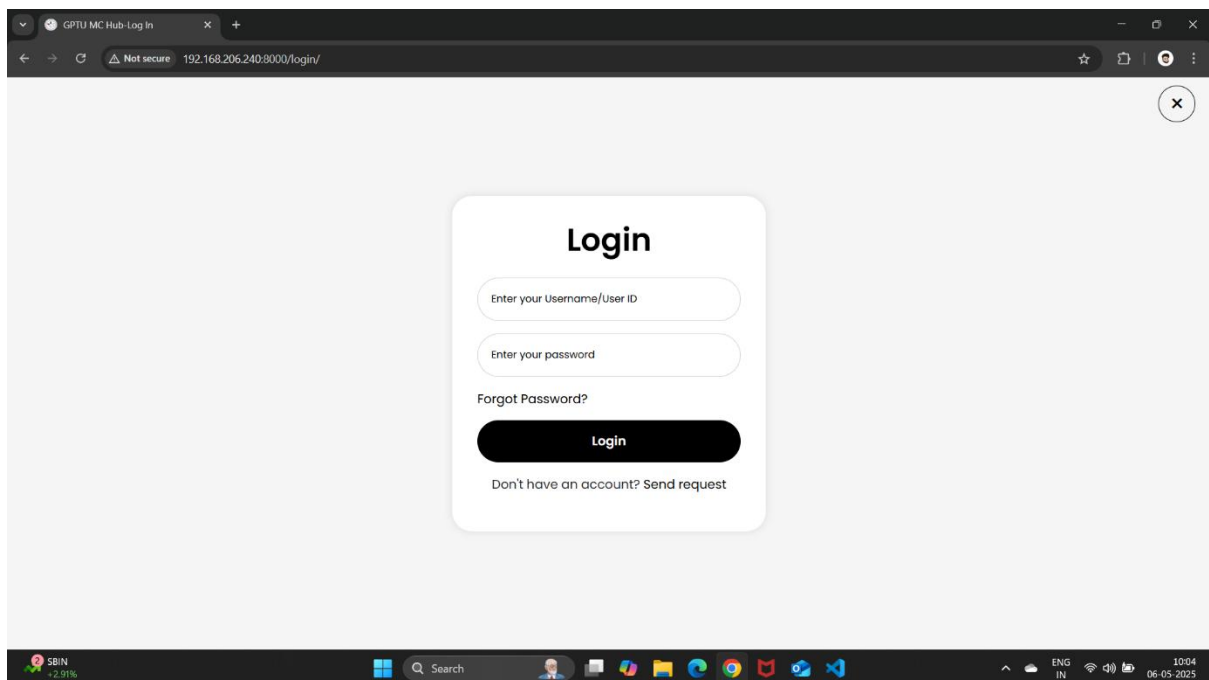
```

11. RESULT EVALUATION

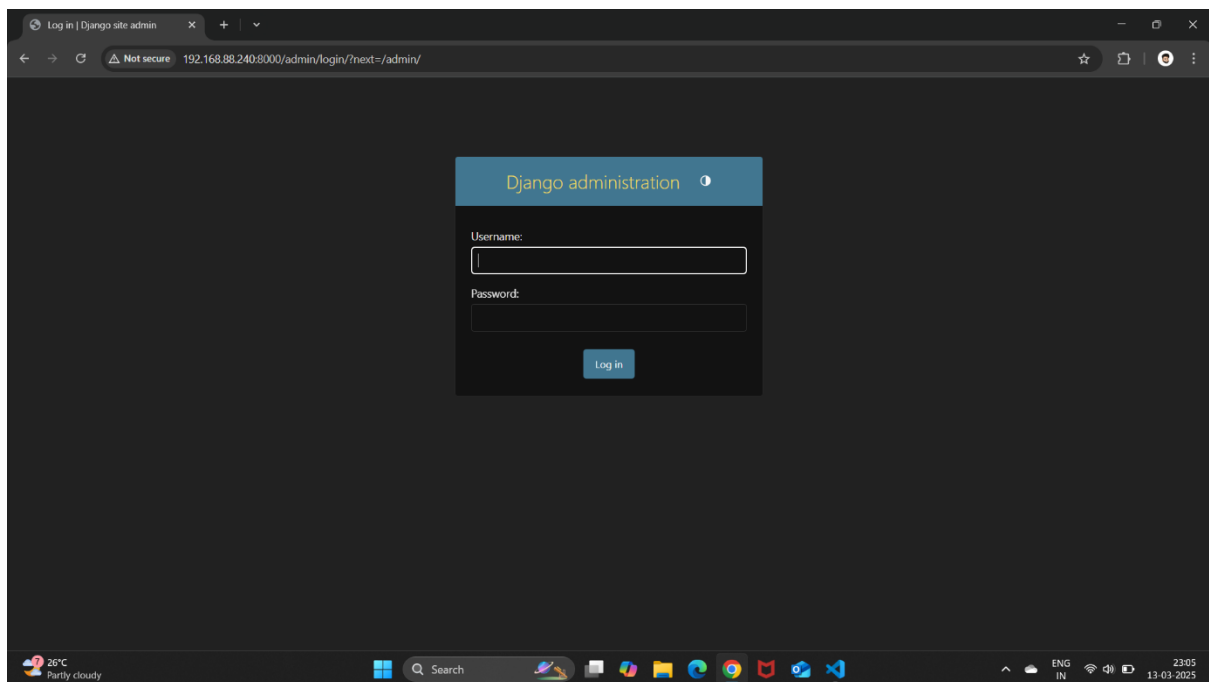
Home Page:



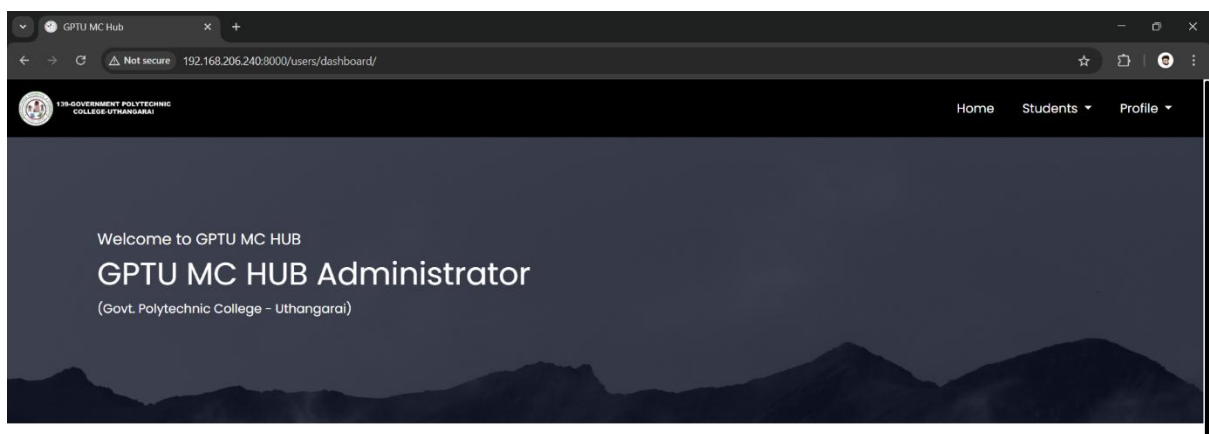
Login Page:



Admin Panel Login Page:



User's Home Page:



GPTU Mentor & Counselling Hub

Welcome to the **GPTU Mentor & Counselling Hub (GPTU MC HUB)** - a comprehensive platform designed to streamline student data management efficiently. Our Web application provides an intuitive and secure way to handle student records, ensuring accurate data storage, easy retrieval, and seamless access for administrators, faculty, and students.

Our Mission

Our goal is to simplify academic data management by offering a structured and user-friendly system that enables institutions to maintain student details, track academic performance, and manage essential records with ease.

Key Features



12. FUTURE SCOPE

The **GPTU Mentor & Counselling Hub** has strong potential for future improvements and new features to make the system even more useful and efficient. Some possible future enhancements include:

- **SMS/Email Notifications:** Automatically send alerts to students and staff for updates, feedback responses, or counselling schedules.
- **Mobile App Integration:** Develop an Android/iOS app for easier access on mobile devices.
- **Real-Time Chat System:** Add a chat feature for live communication between students and staff or mentors.
- **AI-Based Suggestions:** Use AI to analyse student data and suggest personalized counselling or academic support.
- **Analytics Dashboard:** Show detailed reports and graphs on student performance, feedback trends, and support activity.
- **Multi-Institute Support:** Extend the system to be used by multiple colleges or departments under a university.

These updates will make the system more interactive, user-friendly, and powerful in supporting student success.

13. CONCLUSION

The **GPTU Mentor & Counselling Hub** is a useful web application designed to make student data management and counselling support easier for colleges. It brings all important student information—like personal, academic, and non-academic records—into one secure and easy-to-use platform. With this system, staff can manage students more effectively, give timely support, and reduce the need for paperwork.

Built using Django, the system is reliable, scalable, and future-ready. It improves communication between students and staff while helping institutions maintain organized records. Overall, this project aims to support student success by providing a smart and digital way to handle student services in a college environment.

14. REFERENCES

- **"Web Technologies: HTML, JavaScript, PHP, Java, JSP, ASP.NET, XML and Ajax"**
Author: Uttam K. Roy
Publisher: Oxford University Press
- **"Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics"**
Author: Jennifer Niederst Robbins
Publisher: O'Reilly Media
- **"Two Scoops of Django 3.x: Best Practices for the Django Web Framework"**
Authors: Audrey Roy Greenfeld, Daniel Roy Greenfeld
Publisher: Two Scoops Press
- **"Beginning Django: Web Application Development and Deployment with Python"**
Author: Nigel George
Publisher: Apress
- **"Database System Concepts"**
Authors: Abraham Silberschatz, Henry F. Korth, S. Sudarshan
Publisher: McGraw-Hill Education
- **"Python Crash Course: A Hands-On, Project-Based Introduction to Programming"**
Author: Eric Matthes
Publisher: No Starch Press